

Lab Exercise 2: State Machines

1. At this point your tutor will discuss the concept of state machines with you. Python has a state machine library now that you will want to look at the following reading:



<https://buildmedia.readthedocs.org/media/pdf/python-statemachine/stable/python-statemachine.pdf>

https://www.python-course.eu/finite_state_machine.php

https://medium.com/@brianray_7981/tutorial-write-a-finite-state-machine-to-parse-a-custom-language-in-pure-python-1c11ade9bd43

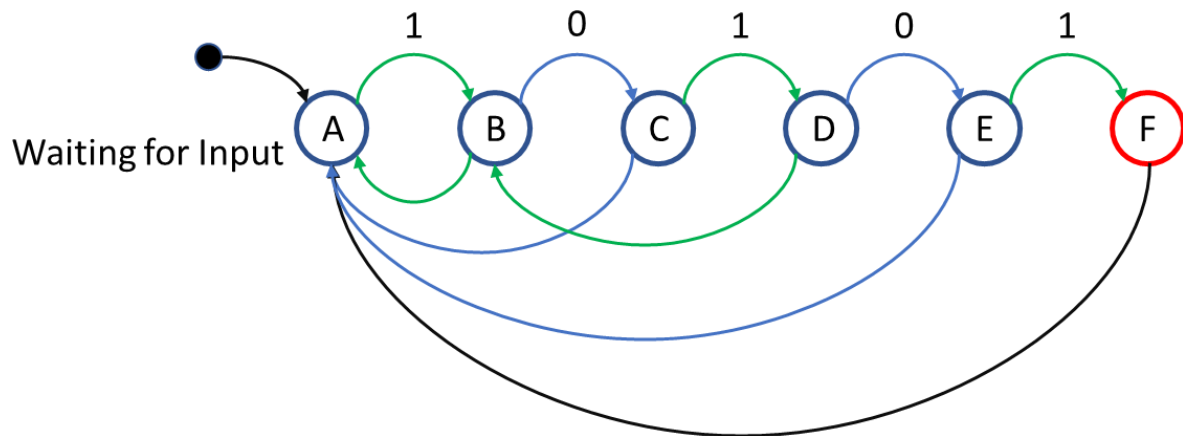
State Machines

(https://en.wikipedia.org/wiki/Finite-state_machine)

Consider a system to track digit input – we are looking for a 5 digit input like a pin code: We could do a sliding window input and look at each 5 digit section of input, or do something more complex like a regex. In this way we have to check each piece of the input against all of the values – we maintain a history of where we are, but not what we have done.

1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0
1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0
1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0
1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0
1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0
1	0	0	1	0	1	0	1	1	1	0	1	1	0	0	0

A state machine by contrast maintains some concept of what we have done, and where we are – but only in so far as it is relevant to our functionality. So if we are looking for input we can just keep track of where we should be in the sequence and not what came before.



So considering the state machine above – we could represent this in python as:

```

2. while True:
    inputvar = input("Please enter a digit")
    if currentstate == "W":
        DoStateW()
        previousstate = currentstate

        if inputvar == "1":
            currentstate = "A"
        else:
            currentstate = "W"
    elif currentstate == "A":
        DoStateA()
        previousstate = currentstate

        if inputvar == "0":
            currentstate = "B"
        else:
            currentstate = "W"
  
```

This resembles the switch statements in C style languages – we could of course exploit Pythonic structures to do this as a dictionary (<https://data-flair.training/blogs/python-switch-case/>), or use the State Machine class (<https://python-3-patterns-idioms-test.readthedocs.io/en/latest/StateMachine.html>) that exists in Python 3.

Modifying a state machine is easy – we can add new states and transitions easily and only define their linkages, we don't have to alter the check code or regex that's implementing things for us. The power of state machines is really expressed well in systems like SMTP when we have a subset of inputs that we can allow at any given time and we want to control these – we reduce our tests and complexity by implementing state.



Redraw the state machine to handle a 6th digit.
How many changes did you need to make to your diagram and code?
Explore StateFunction.zip to try out stateful client-server interaction.

Reading Links:



<https://www.smashingmagazine.com/2018/01/rise-state-machines/>

<https://blog.markshead.com/869/state-machines-computer-science/>

<https://www.youtube.com/watch?v=lffJJbqt1fk>

<https://www.youtube.com/watch?v=-Yicg2TTMPs>

3. Once you have the program able to handle repeatedly sending data and the exit condition implement the number guessing game. The key to this exercise is that you will need to parse out what the user is sending you – whether it is a guess or a command. Your server will need to maintain some form of state to handle returning the number of guesses.
4. Extend this exercise to Hangman instead of the number guessing game, at this point you likely want to consider adding some state on both the client side and the server side rather than simply on the server side – why even send a guess if you know it can't be valid.